



ACTIVITAT AVALUABLE AC3

Mòdul: MÒDUL PROPI - Big Data

U3

Professor: Albert Guardiola

Data d'entrega: 4/5/25

Mètode d'entrega: Clickedu.

Instruccions: --

Resultats de l'aprenentatge:

RA3

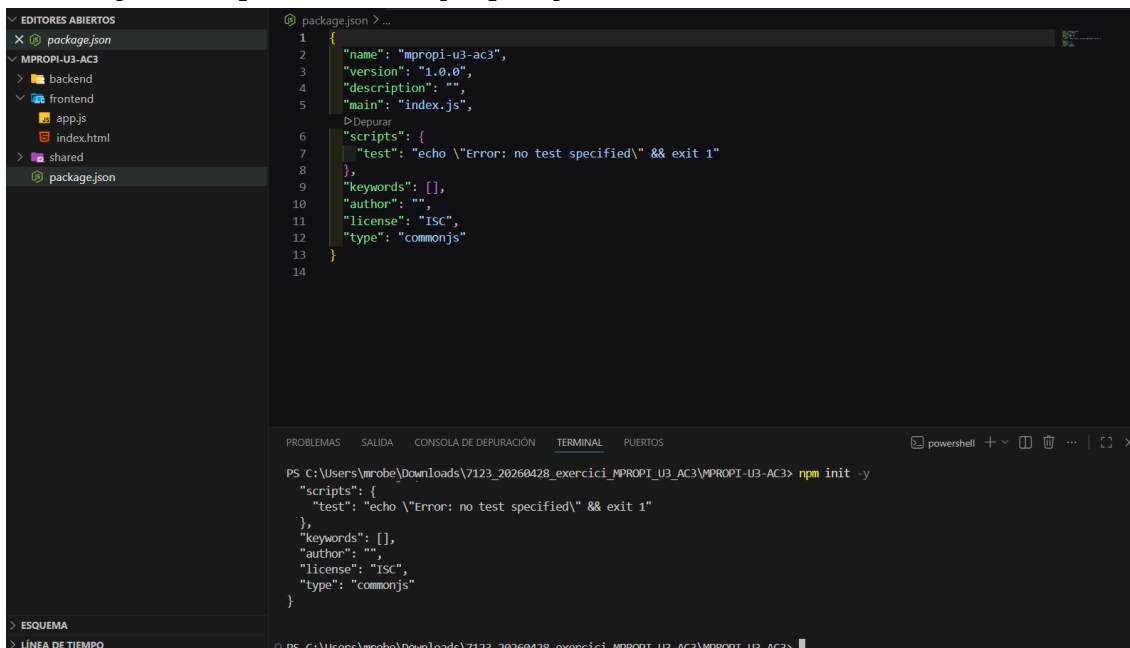
PRÁCTICA GUIADA: COPILOT PROFESSIONAL (PARTE 2 – TESTING CON IA)

REQUISITOS PREVIOS

Intalar dependencias para test

```
npm init -y
```

```
npm install -D vitest jsdom @testing-library/dom  
@testing-library/user-event playwright
```



The screenshot shows the Visual Studio Code editor interface. On the left, the Explorer sidebar shows a project structure with folders 'backend' and 'frontend', and files 'app.js', 'index.html', and 'shared'. The main editor area displays the 'package.json' file with the following content:

```
1 {  
2   "name": "mpropi-u3-ac3",  
3   "version": "1.0.0",  
4   "description": "",  
5   "main": "index.js",  
6   "scripts": {  
7     "test": "echo \\\"Error: no test specified\\\" && exit 1"  
8   },  
9   "keywords": [],  
10  "author": "",  
11  "license": "ISC",  
12  "type": "commonjs"  
13 }  
14
```

Below the editor, the TERMINAL panel is open, showing the output of the command 'npm init -y':

```
PS C:\Users\mrobe\Downloads\7123_20260428_exercici_MPROPI_U3_AC3> npm init -y  
"scripts": {  
  "test": "echo \\\"Error: no test specified\\\" && exit 1"  
},  
"keywords": [],  
"author": "",  
"license": "ISC",  
"type": "commonjs"  
}
```



The screenshot shows a VS Code editor with a project named 'MPROPI-U3-AC3'. The file explorer on the left shows the project structure, including 'package.json'. The main editor displays the content of 'package.json', which includes fields for name, version, description, main, scripts, keywords, author, license, type, and devDependencies. The 'scripts' field is currently set to a test command. The terminal at the bottom shows the execution of 'npm init -y' and 'npm install -D vitest jsdom @testing-library/dom', followed by a message about funding and a list of vulnerabilities found.

```
1 {
2   "name": "mpropi-u3-ac3",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \\Error: no test specified\\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "type": "commonjs",
13  "devDependencies": {
14    "@testing-library/dom": "^10.4.1",
15    "jsdom": "^29.1.1",
16    "vitest": "^4.1.5"
17  }
18 }
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\mrobe\Downloads\7123_20260428_exercici_MPROPI_U3_AC3\MPROPI-U3-AC3> npm init -y

PS C:\Users\mrobe\Downloads\7123_20260428_exercici_MPROPI_U3_AC3\MPROPI-U3-AC3> npm install -D vitest jsdom @testing-library/dom

added 96 packages, and audited 97 packages in 14s

26 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

PS C:\Users\mrobe\Downloads\7123_20260428_exercici_MPROPI_U3_AC3\MPROPI-U3-AC3>

Añadir al proyecto (package.json) los scripts de test

```
"scripts": {
  "test": "vitest",
  "test:ui": "vitest --ui",
  "test:e2e": "playwright test"
}
```



```
package.json X
package.json > {} scripts
1  {
2    "name": "mprop1-u3-ac3",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "vitest",
8      "test:ui": "vitest --ui",
9      "test:e2e": "playwright test"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "type": "commonjs",
15   "devDependencies": {
16     "@testing-library/dom": "^10.4.1",
17     "@testing-library/user-event": "^14.6.1",
18     "jsdom": "^29.1.1",
19     "playwright": "^1.59.1",
20     "vitest": "^4.1.5"
21   }
22 }
```

A. TESTING UNITARIO

Fragmento teórico

El **test unitario** verifica una unidad aislada de código (normalmente una función).

Características:

- no depende del DOM ni del backend
- ejecución rápida
- detecta errores lógicos
- fácil de automatizar

Ejemplo típico:

- validar inputs
- calcular valores
- comprobar condiciones



Limitación clave:

- no detecta errores de integración (frontend/backend)

EJERCICIO A1. Test unitario básico

Vamos a testear la función *validateLogin* en `shared/validation.js`

Acción

Crear `validation.test.js`:

```
import { describe, it, expect } from 'vitest'
import { validateLogin } from './validation'

describe('validateLogin', () => {
  it('fails if email is empty', () => {
    expect(validateLogin('', '12345678').valid).toBe(false)
  })

  it('fails if password is empty', () => {
    expect(validateLogin('test@test.com', '').valid).toBe(false)
  })

  it('passes with valid input', () => {
    expect(validateLogin('test@test.com', '12345678').valid).toBe(true)
  })
})
```

```
package.json X validation.test.js X
shared > validation.test.js > ...
1  import { describe, it, expect } from 'vitest'
2  import { validateLogin } from './validation'
3
4  describe('validateLogin', () => {
5    it('fails if email is empty', () => {
6      expect(validateLogin('', '12345678').valid).toBe(false)
7    })
8
9    it('fails if password is empty', () => {
10     expect(validateLogin('test@test.com', '').valid).toBe(false)
11   })
12
13   it('passes with valid input', () => {
14     expect(validateLogin('test@test.com', '12345678').valid).toBe(true)
15   })
16 })
17
```



Cómo ejecutar

```
npm run test
```

(Se va a ejecutar el script `test` del `package.json` (comando `vitest`) sobre todos los archivos `*.test` que se encuentren en el proyecto.)

Qué observar

- tests rápidos
 - resultado inmediato
 - lógica validada
-

EJERCICIO A2. Edge cases con IA

Teoría breve

Un **edge case** es un caso límite:

- valores vacíos
- null / undefined
- formatos incorrectos

Son donde suelen aparecer bugs.

Acción

Fija el scope del agente en `validate.test.js` y dale el prompt:

```
añade tests unitarios con edge cases
```

Observa cómo se modifica el archivo de test con nuevos casos de test.



→ detectar errores automáticamente al ejecutar

Acción

Vamos a modificar la función `validateLogin` para romper el código y comprobar que ya no se pasan los tests unitarios.

```
if (!email && !password)
```

```
shared > js validation.js > ...  
1  export function validateLogin(email, password) {  
2    if (!email && !password) {  
3      return { valid: false, error: 'Campos obligatorios' }  
4    }  
5  
6    // Validar que no sean solo espacios en blanco  
7    if (typeof email === 'string' && !email.trim()) {  
8      return { valid: false, error: 'Email no puede estar vacío' }  
9    }  
10  
11    if (typeof password === 'string' && !password.trim()) {  
12      return { valid: false, error: 'Contraseña no puede estar vacía' }  
13    }  
14  
15    return { valid: true }  
16  }  
17  
18
```

Ejecutar

```
npm run test
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  
  
100|      expect(validateLogin(0, '12345678').valid).toBe(false)  
  
Test Files  1 failed (1)  
  Tests    4 failed | 18 passed (22)  
Start at   15:56:14  
Duration   34ms  
  
FAIL Tests failed. Watching for file changes...  
press h to show help, press q to quit
```

Resultado

- tests fallan



- bug detectado

Acción

Recuerda volver a dejar la función *validateLogin* en su forma correcta.



B. TEST FUNCIONAL (SIMULACIÓN DE DEPENDENCIAS)

Fragmento teórico

El **test funcional** verifica cómo se comporta el sistema desde el punto de vista del usuario.

Características:

- interactúa con el DOM
- simula acciones (click, escribir)
- valida comportamiento visible

Ejemplo:

- escribir en input
- hacer click
- ver mensaje

Diferencia con unitario:

- unitario → lógica interna
 - funcional → comportamiento externo
-

EJERCICIO B1. Test funcional básico (frontend)

Acción

Crea el archivo

`frontend/app.test.js`

con el siguiente contenido

```
import userEvent from '@testing-library/user-event'
import { test, expect } from 'vitest'

test('user interaction', async () => {
  // 1. Crear DOM
  document.body.innerHTML = `
    <input id="email" />
    <input id="password" />
    <button id="loginBtn">Login</button>
    <p id="message"></p>
  `
  // 2. Simular interacción
  const user = userEvent.setup()
  user.type('#email', 'test@example.com')
  user.type('#password', 'password123')
  user.click('#loginBtn')
  // 3. Verificar resultado
  expect(document.querySelector('#message').textContent).toContain('Login exitoso')
})
```



```
const email = document.getElementById('email')
const password = document.getElementById('password')
const button = document.getElementById('loginBtn')

await userEvent.type(email, 'test@test.com')
await userEvent.type(password, '12345678')
await userEvent.click(button)
})
```

Cómo ejecutar

npm run test

```
frontend > app.test.js > ...
4 test('user interaction', async () => {
5   // 1. Crear DOM
6   document.body.innerHTML = `
7     <input id="email" />
8     <input id="password" />
9     <button id="loginBtn">Login</button>
10    <p id="message"></p>
11  `
12
13   const email = document.getElementById('email')
14   const password = document.getElementById('password')
15   const button = document.getElementById('loginBtn')
16
17   await userEvent.type(email, 'test@test.com')
18   await userEvent.type(password, '12345678')
19   await userEvent.click(button)
20 })
21
```

RERUN shared/validation.test.js x2

```
> frontend/app.test.js (1 test | 1 failed) 4ms
  x user interaction 3ms
✓ shared/validation.test.js (22 tests) 4ms
  ✓ validateLogin (22)
```

Recuerda que Vitest detecta automáticamente:

- archivos *.test.js
- dentro de cualquier carpeta



Qué observar

! Problema muy importante

Este test **no funcionará completamente** por varios motivos:

1. `app.js` usa `fetch`: hay que simular ese proceso para poder hacer el test, pero al estar simulando sólo el frontend, **no hay backend real**. Debemos hacer un mock del `fetch`.
2. **No hay en `app.js` ninguna función testeable**, para ello hay que modificar el archivo así para exportar la función:

```
export function initApp() {
  document.getElementById('loginBtn').addEventListener('click',
  async () => {
    const email = document.getElementById('email').value
    const password = document.getElementById('password').value

    const response = await fetch('http://localhost:3000/login',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    })

    const data = await response.json()
    document.getElementById('message').textContent =
    data.message
  })}
```

3. Al estar ejecutando el test en Node y no en el navegador, **NO HAY DOM**. Debemos pedirle a Vitest que lo simule. Para ello, añadimos el archivo `vitest.config.js` a la raíz del proyecto, con este contenido:

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    environment: 'jsdom'
  }
})
```

4. Aplicados estos cambios, el test `app.test.js` queda así:

```
// 0. Importamos JSDOM para simular DOM
// @vitest-environment jsdom
import { test, expect, vi } from 'vitest'
import userEvent from '@testing-library/user-event'
import { initApp } from './app.js'

test('login correcto', async () => {
  // 1. Crear DOM
  document.body.innerHTML = `
```



```

    <input id="email" />
    <input id="password" />
    <button id="loginBtn">Login</button>
    <p id="message"></p>
  `

// 2. Inicializar app
initApp()

// 3. Mock fetch
global.fetch = vi.fn(() =>
  Promise.resolve({
    json: () => Promise.resolve({ message: 'Login correcto' })
  })
)

// 4. Simular usuario
await userEvent.type(document.getElementById('email'),
'test@test.com')
await userEvent.type(document.getElementById('password'),
'12345678')
await userEvent.click(document.getElementById('loginBtn'))

// 5. Verificar

expect(document.getElementById('message').textContent).toBe('Login
correcto')
})

```

👉 Ahora sí que el test debe pasarse exitosamente.

Esto nos sirve para ver que:

- escribir test $\neq$ test válido
- falta **simular dependencias**

```

RERUN frontend/app.test.js x1

✓ frontend/app.test.js (1 test) 370ms
✓ login correcto 369ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 16:06:11
Duration 537ms

PASS Waiting for file changes...

```



EJERCICIO B2. Error de usuario

Teoría breve

Los tests funcionales deben cubrir:

- casos correctos
- casos de error

Acción

Con el contexto fijado en *app.test.js*, escribe el prompt:

genera test donde login falla

```
test('login falla', async () => {
  document.body.innerHTML = `
    <input id="email" />
    <input id="password" />
    <button id="loginBtn">Login</button>
    <p id="message"></p>
  `

  initApp()

  global.fetch = vi.fn(() =>
    Promise.resolve({
      json: () => Promise.resolve({ message: 'Credenciales incorrectas' })
    })
  )

  await userEvent.type(document.getElementById('email'), 'wrong@test.com')
  await userEvent.type(document.getElementById('password'), 'wrongpass')
  await userEvent.click(document.getElementById('loginBtn'))

  expect(document.getElementById('message').textContent).toBe('Credenciales incorrectas')
})
```

Ejecutar

npm run test

Observa que pasa el test.



```
RERUN frontend/app.test.js x3
```

```
✓ frontend/app.test.js (2 tests) 888ms
```

```
✓ login correcto 448ms
```

```
✓ login falla 437ms
```

```
Test Files 1 passed (1)
```

```
Tests 2 passed (2)
```

```
Start at 16:07:49
```

```
Duration 1.06s
```



C. TEST END-TO-END (E2E)

Fragmento teórico

El **test E2E (end-to-end)** prueba el sistema completo:

- frontend
- backend
- red

Simula un usuario real en un navegador.

Características:

- más lento
- más realista
- detecta errores de integración

Ejemplo:

- login completo
 - navegación
 - flujo de usuario
-

EJERCICIO C1. Error de usuario

Acción:

Prompt al agente:

Start the backend server, open the frontend in a browser, and test the login flow with three different credential pairs:

1. correct credentials (test@test.com / 12345678)
2. wrong password
3. empty fields

Verify the UI message for each case and report the results.
Show the interaction with an integrated browser.

Observa cómo:

- El agente te pide permiso para instalar dependencias faltantes del servidor (express).



- El agente te pide permiso para instalar (si es la primera vez) las tools necesarias para el test E2E (chromium, etc).
- El agente levanta la app en el localhost.
- El agente abre un navegador integrado en el IDE (chromium). LOL.
- El agente simula la interacción con la app web (DOM) y lee la respuesta del servidor.
- Se presentan los resultados del test.



D. REVIEWS (AUDITORÍAS)

EJERCICIO D1. Review de seguridad

Prompt al **chat**:

@workspace Review security issues

? Brechas esperadas:

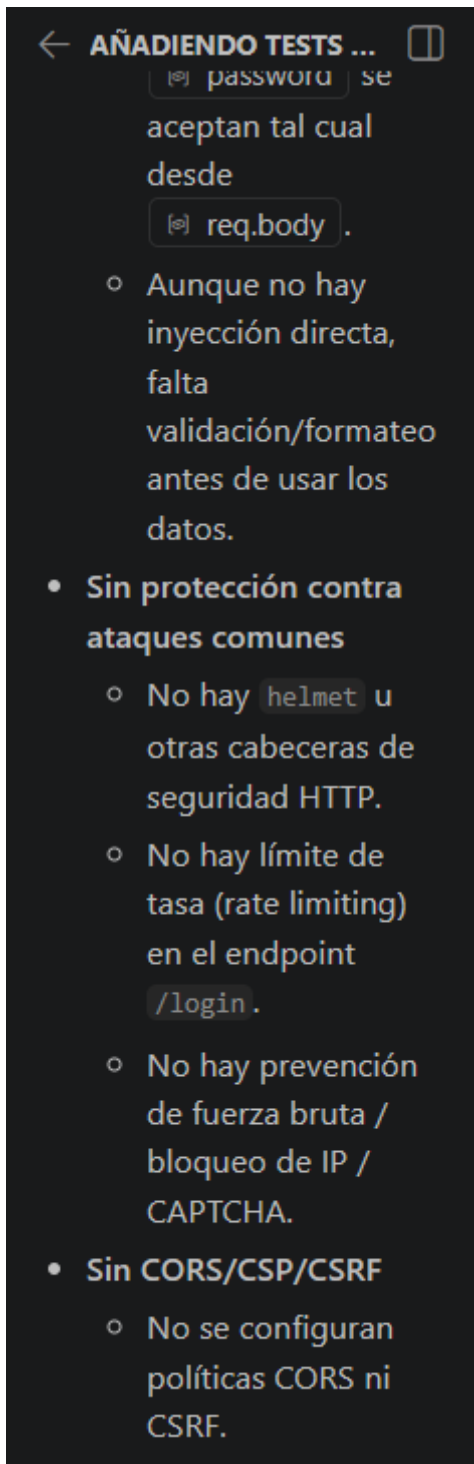
- passwords harcodeadas en el código ✗
- CORS no configurado ✗
- No hay HTTPS ✗
- No hay *rate limiting* en los endpoints del servidor ✗
- (...)



Revisión de seguridad del workspace

1. Backend (`server.js`)

- **Credenciales hardcodeadas**
 - El usuario y la contraseña están en el código (`test@test.com` / `12345678`).
 - Esto no es seguro en ningún ambiente real.
- **Sin hashing ni almacenamiento seguro**
 - No hay base de datos, pero si esto fuera real, no hay hashing/pepper/salt para contraseñas.
- **Sin validación de entrada**
 - `email` y `password` se



EJERCICIO D2. Review de accesibilidad

Opción 1 (bàsica)

@file index.html Review accessibility (WCAG)



Opción 2 (refinada con *agent skills*)

Objectiu

Dar **contexto externo estructurado (skills)** a Copilot

Idea clau

Copilot sense context:

- superficial

Copilot amb context estructurat:

- molt millor

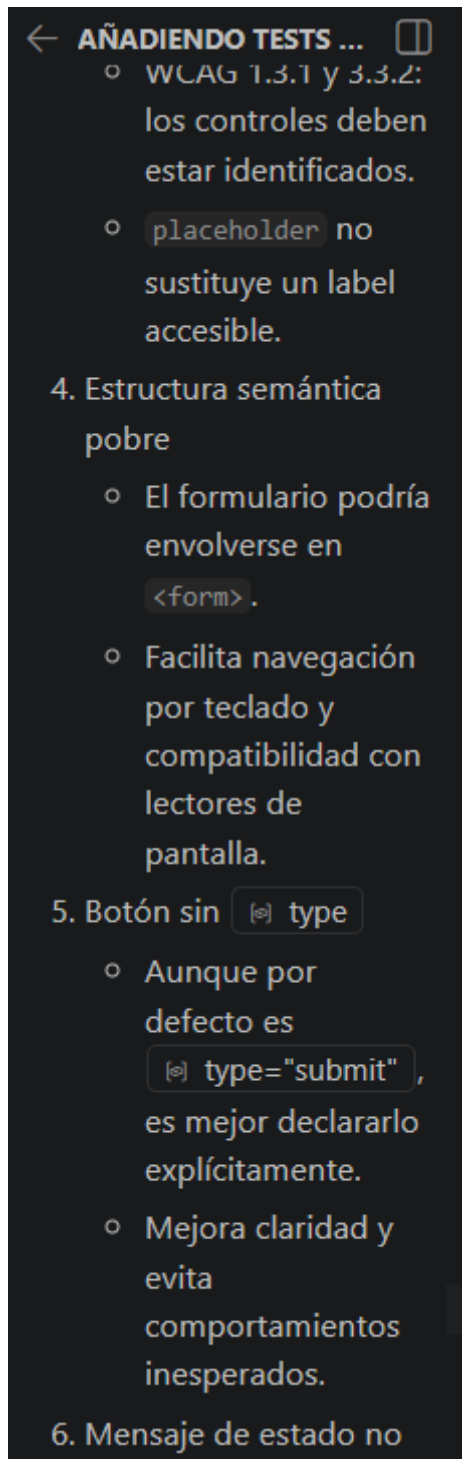


Revisión de accesibilidad de

 index.html

Problemas detectados

1. `html` sin `lang`
 - Debe tener `<html lang="es">` o el idioma correcto.
 - WCAG 3.1.1: especificar idioma del documento.
2. Falta la sección `<head>` y `<title>`
 - No hay metadata básica ni título de página.
 - WCAG 2.4.2: título de página descriptivo.
3. Inputs sin `label`
 - Los campos usan `placeholder`, pero no etiquetas `<label>`.
 - WCAG 1.3.1 y 3.3.2:



PAS A PAS — CREAR FITXER DE “SKILLS WCAG”

1. Crear fitxer

 wcag-guidelines.md

2. Generar skills con Copilot (chat)



PROMPT: Generate a concise developer-friendly WCAG checklist for HTML forms

3. Usar aquest fitxer com a context

👉 Selecciona el contingut del fitxer (ratolí) i copiar-lo en el fichero **wcag-guidelines.md**

4. Prompt amb context

PROMPT: Apply these WCAG guidelines to index.html

6. Alternativa amb referència

PROMPT: @file wcag-guidelines.md @file index.html Improve accessibility

- Compara els resultats del review sense i amb skills.

The screenshot shows a VS Code editor with a file named `wcag-guidelines.md`. The file contains a checklist for HTML forms, including requirements like using `<html lang="es">` and `<head>` with `<title>`, including `<form>` around input fields, using `<label for="...">` for each input, select, or text area, adding names to all form controls, not relying solely on placeholders for identifying fields, marking buttons with `type="submit"` or `type="button"`, providing clear contextual help (e.g., `aria-describedby` for additional descriptions), validating and displaying accessible errors (e.g., `aria-invalid="true"` for error fields, `aria-describedby` linked to error text), announcing dynamic changes with `role="status"` or `aria-live="polite"`, maintaining logical and coherent tab order, ensuring sufficient contrast in labels, placeholders, and buttons, allowing full keyboard interaction, avoiding messages that depend solely on color, using `type="email"`, `type="password"`, and `autocomplete` when appropriate, and clearly indicating required fields in labels or with visible text.

Below the checklist, there is a section titled `## Estructura básica` with a list of items to be added to the `index.html` file:

- Usar `<html lang="es">` y `<head>` con `<title>`
- Incluir `<form>` alrededor de los campos de entrada
- Usar `<label for="...">` para cada input, select o textarea
- Añadir `name` a todos los controles de formulario

At the bottom of the editor, the `TERMINAL` tab shows the output of a test command:

```
Test Files 2 passed (2)
Tests 24 passed (24)
```

On the right side of the editor, there is a sidebar with a section titled `ANADIENDO TESTS ...`. It contains a list of tests and a prompt to improve accessibility:

@file wcag-guidelines.md @file index.html Improve accessibility

Below this, there is a section titled `¿Ejecutar el comando push?` with a `npm test -- frontend/app.test.js` command and a `powerShell` button.



```
package.json  localhost:3000  validation.test.js  app.test.js  wcag-guidelines.md  index.html  ...
wcag-guidelines.md > ...
1 WCAG Checklist para formularios HTML
2
3 ## Estructura básica
4 - Usar `<html lang="es">` y `<head>` con `<title>`
5 - Incluir `<form>` alrededor de los campos de entrada
6 - Usar `<label for="...">` para cada input, select o textarea
7 - Añadir `name` a todos los controles de formulario
8 - No depender solo de `placeholder` para identificar campos
9
10 ## Tipos de input y atributos
11 - Marcar botones con `type="submit"` / `type="button"`
12 - Usar campos `type="email"`, `type="password"` y `autocomplete` cuando aplique
13 - Si hay campos obligatorios, indicarlo claramente en la etiqueta y/o con texto visible
14
15 ## Ayuda contextual y validación
16 - Proveer ayuda contextual clara (`aria-describedby` para descripciones adicionales)
17 - Validar y mostrar errores accesibles:
18   - `aria-invalid="true"` en campos con error
19   - `aria-describedby` enlazado con texto de error
20 - Anunciar cambios dinámicos con `role="status"` o `aria-live="polite"`
21
22 ## Navegación y interacción
23 - Mantener el orden de tabulación lógico y coherente
24 - Permitir interacción completa con teclado
25
26 /login-incorrecto 554ms
```